

Endocrine Atlas — Implementation Prompt

SUMMARY

You are the primary coding agent responsible for implementing a small educational web application. Build the project end-to-end. Do not over-engineer it. The application is a junior-high-school-level interactive endocrine gland anatomy viewer.

TECHNICAL SKILLS

Vite
React
TypeScript
Three.js
React Three Fiber
Drei
[@react-three/fiber](#)
[@react-three/drei](#)

PRODUCT GOAL

Build a web page similar in interaction style to [thebuggeddev/anatomy](#), but specifically for the human endocrine glands. Main interaction: Select gland → camera focuses on gland → gland is highlighted → user can horizontally rotate the 3D view → knowledge card appears on the right.

SCOPE CONSTRAINT

1.5-week MVP with strict non-goals: no medical-grade realism, no backend/database/auth, no AI chatbot, no complex simulations, no unnecessary dependencies or advanced rendering systems.

ASSET STRATEGY

Do not depend on external 3D human anatomy assets. Construct the body using simple Three.js primitives as a semi-transparent mannequin/silhouette; glands should be visually prominent.

MVP GLANDS

Implement exactly seven first: 下丘脑 Hypothalamus, 垂体 Pituitary, 甲状腺 Thyroid, 肾上腺 Adrenal glands, 胰腺 Pancreas, 卵巢 Ovaries, 睾丸 Testes.

INTERACTION REQUIREMENTS

Every gland clickable; selection highlights gland, dims others, smoothly focuses camera (500–900ms), keeps OrbitControls functional, supports horizontal rotation and zoom, and includes reset view behavior.

EDUCATIONAL CONTENT REQUIREMENTS

Audience: 初中生. Chinese-primary knowledge cards with English secondary labels. Each card includes location, hormones (1–2), and basic functions (1–3), avoiding advanced endocrinology details.

UI/LAYOUT REQUIREMENTS

Desktop-first split layout with 3D viewer (~65–70%) and information card (~30–35%), clear initial instructions, and usable stacked layout below ~768px.

ENGINEERING ARCHITECTURE

Data-driven gland content in `src/data/glands.ts`; simple React state for selected gland; isolated camera focus hook (e.g., `useCameraFocus`); consistent coordinate system for anatomical positioning; avoid giant monolithic `App.tsx`.

RECOMMENDED PROJECT STRUCTURE

`src/components` (AnatomyScene, BodyModel, Gland, GlandSystem, KnowledgeCard, Header, ControlsHint), `src/data/glands.ts`, `src/hooks/useCameraFocus.ts`, `src/types/gland.ts`, `App.tsx`, `main.tsx`, `index.css`.

DEVELOPMENT WORKFLOW

Phase order: 1) Scene, 2) Glands, 3) Interaction, 4) Rotation, 5) Knowledge Card, 6) Polish.

DELIVERY PLAN

Day 1 setup; Day 2 mannequin; Day 3 seven glands; Day 4 click/selection/highlight; Day 5 camera focus/reset; Day 6 OrbitControls tuning; Day 7 knowledge data/card; Day 8 UI/layout/reset; Day 9 polish; Day 10 QA and bug fixes.

DEFINITION OF DONE

3D mannequin and seven glands rendered and positioned; all glands clickable with highlight and smooth focus; rotation/zoom constraints and reset work; complete Chinese+English educational cards; polished desktop and usable small-screen UI; TypeScript/build health; no unnecessary backend/API/large assets.

FINAL PRODUCT PHILOSOPHY

Primary flow: LOOK → CLICK → FOCUS → ROTATE → LEARN. Prioritize working interaction > anatomical realism > feature count.